

Data frames and loading data

KING'S
College
LONDON

Social and Cultural Analytics (7AAVBCS5)

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

2026-10-12



Lecture Plan

- **Part I: This week's London data**
- **Part II: Data frames**
 - Creating data frames
 - Structure and properties
- **Part III: Files and loading CSVs**
 - Where your files are
 - What a CSV is
 - Reading CSV files
- **Part IV: Selecting rows and columns**
 - `loc` and `iloc`
 - Conditional selection
 - Saving filtered or derived data
- **Part V: Live coding**

Key Learning Outcomes

By the end of this week you will be able to:

- Create and manipulate data frames in Python
- Select from data frames using several techniques
- Understand how file paths work
- Read data from external CSV files
- Write data frames to CSV files
- Apply these skills to real datasets

London Data

The question

From this week onwards we will be trying to look at a unified subject as a way to tie the teaching and our understanding together. That subject will be... London!

Specifically today we will be asking:

What can borough-level and venue-level data tell us about London’s cultural infrastructure?

We will use two linked datasets:

File	One row is...	Use today	Source
London Datastore’s Ward Profiles and Atlas	one London borough	meaningful numeric summaries	Link
London Datastore’s Cultural Infrastructure Map 2023	one cultural venue	raw venue-level data	Link

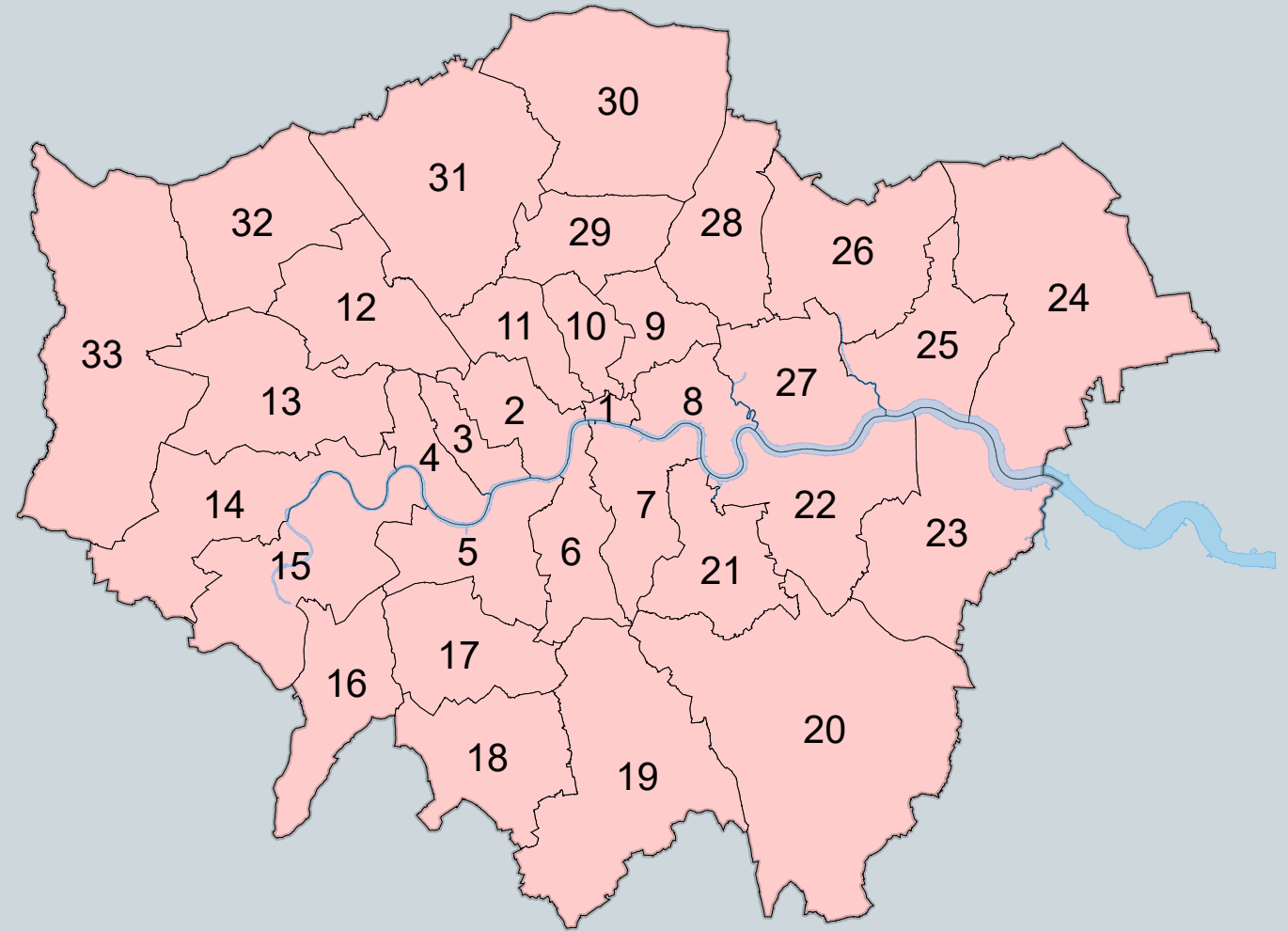
London and its boroughs

Greater London

- the city-region used for London-wide government
- made up of **32 boroughs** plus the **City of London**
- boroughs are local government areas, not neighbourhoods
- each borough contains many wards, streets, and named places

Why boroughs today?

- cultural venue data records borough names and codes
- boroughs can be linked to population, area, income, and transport data
- this makes comparisons possible
- but boroughs also hide variation inside them



The borough context file

```
1 import pandas as pd
2
3 borough_context = pd.read_csv(
4     "../..data/london_cultural_infrastructure_2023_borough_context.csv"
5 )
6
7 borough_context[[
8     "borough_name",
9     "venues",
10    "population_2015",
11    "venues_per_10k_residents",
12    "venues_per_sq_km",
13    "median_household_income_2012_13",
14 ]].head()
```

	borough_name	venues	population_2015	venues_per_10k_residents	venues_per_sq_km	median_household_income_2012_13
0	Westminster	653	230550.0	28.32	29.68	80760.0
1	Camden	469	230300.0	20.36	21.51	67990.0
2	Hackney	403	262700.0	15.34	21.10	42690.0
3	Southwark	307	308650.0	9.95	10.27	48000.0
4	Tower Hamlets	295	284300.0	10.38	13.66	45720.0

The cultural infrastructure file

```
1 import pandas as pd
2
3 cultural_infrastructure = pd.read_csv(
4     "../..data/london_cultural_venues_2023.csv"
5 )
6
7 cultural_infrastructure[[
8     "name",
9     "venue_type",
10    "borough_name",
11    "website",
12    "latitude",
13    "longitude",
14 ]].head()
```

	name	venue_type	borough_name	website	latitude	longitude
0	DAGENHAM RM10	Artists workspaces	Barking and Dagenham	https://www.arbeit.org.uk/dagenham	51.549708	0.156633
1	Ice House Court	Artists workspaces	Barking and Dagenham	www.bowarts.org www.artiststudiofinder.org ...	51.531488	0.075994
2	River Mills Studios (Bow Arts)	Artists workspaces	Barking and Dagenham	https://bowarts.org/studios/riverside-mills-st...	51.532525	0.075726
3	Vue Dagenham	Cinemas	Barking and Dagenham	NaN	51.532095	0.135759
4	Bow Arts	Creative workspaces	Barking and Dagenham	www.bowarts.org www.artiststudiofinder.org ...	51.531410	0.077946

Raw vs Aggregated Data

Raw Data

- Individual records
- One row = one person, place, patient, student, venue
- Can group by features (gender, borough, type, etc.)
- Flexible and more useful, but larger

Aggregated/contextual data

- Categorical summaries
- One row = one borough, one degree, one country, one genre of music
- Good for comparisons
- Easier to summarise numerically
- Limited to the groupings already made - made up of data you cannot get back (the individual records)

Quick test: if you see words like “average,” “total,” “percentage by group,” or cross-tabs, you are probably looking at aggregated data.

Raw or aggregated?

borough_name	venues	population_2015	venues_per_10k_residents	median_household_income_2012_13
Westminster	653	230550	28.32	80760
Camden	469	230300	20.36	67990
Hackney	403	262700	15.34	42690
Lewisham	148	293050	5.05	43360

Raw or aggregated?

venue_id	name	venue_type	borough_name	latitude	longitude
cim2023_0628	DAGENHAM RM10	Artists workspaces	Barking and Dagenham	51.5497	0.1566
cim2023_0918	Vue Dagenham	Cinemas	Barking and Dagenham	51.5321	0.1358
cim2023_2740	Barking Library	Libraries	Barking and Dagenham	51.5363	0.0799
cim2023_3090	Broadway Theatre	Theatre	Barking and Dagenham	51.5366	0.0795

Review

What we've learned so far...

- Python as a calculator
- Data types (`int`, `float`, `str`, `bool`)
- How to assign variables
- Manipulating/operating on variables
- Notebook basics
- Lists, indexing and slicing
- arrays (numpy) and array-wise operations
- Basic descriptive stats
 - min, max, sum, mean, median, standard deviation
- Dictionaries
- f-strings
- Comparisons and `if/elif/else`

This week: tables of data, and getting them off your hard drive

Part II: Data Frames

Data Frames: tables in Python

What is a Data Frame?

- pandas' way of storing tabular data
- Like a spreadsheet (e.g., Excel) or database table (e.g., SQL)
- Rows = observations/cases
- Columns = variables/features
- **Two dimensional**, unlike the lists and arrays we have used so far

Key Properties:

- Columns can be different types (strings, numerical, boolean)
- Each column must have the same length (of observations)
- Each column has a name (best when they're meaningful!)
- Can mix numbers, text and True/False

Think of it as: multiple arrays side by side, with names

In Positron, click a data frame in the **Variables** pane to open it as a table.

Creating a data frame

```
1 import pandas as pd
2
3 social_media = pd.DataFrame({
4     "platform": ["Twitter", "Instagram", "TikTok", "Facebook"],
5     "users": [450, 2000, 1000, 3000],
6     "founded": [2006, 2010, 2016, 2004],
7     "public_company": [False, False, False, True],
8 })
9 social_media
```

	platform	users	founded	public_company
0	Twitter	450	2006	False
1	Instagram	2000	2010	False
2	TikTok	1000	2016	False
3	Facebook	3000	2004	True

A **dictionary** of columns: the key is the column name, the value is the list of entries.

`pd.DataFrame(data)` creates a pandas table. Here `data` is a dictionary: column names on the left, equal-length lists of values on the right.

Exploring structure

Question	pandas tool
How many rows and columns?	<code>df.shape</code>
What are the columns called?	<code>df.columns</code>
What types of values are there?	<code>df.info()</code>
What do the first rows look like?	<code>df.head()</code>
What do numeric variables look like?	<code>df.describe()</code>

Methods have brackets, like `df.head()`. Attributes do not, like `df.shape` and `df.columns`. Use `head(n=1)` to inspect one complete row and ask: what single thing does this row describe?

Exploring a real data frame

```
1 borough_context = pd.read_csv(  
2     "../..data/london_cultural_infrastructure_2023_borough_context.csv"  
3 )  
4 borough_context.head()
```

	borough_name	borough_code	venues	venue_types	venues_with_websites	website_share	population_2015	area_sq_km	population_density_2013	media
0	Westminster	E09000033	653	24	578	0.885	230550.0	22.0	10285.8	80760
1	Camden	E09000007	469	26	369	0.787	230300.0	21.8	10482.1	67990
2	Hackney	E09000012	403	26	346	0.859	262700.0	19.1	13471.7	42690
3	Southwark	E09000028	307	25	275	0.896	308650.0	29.9	9976.9	48000
4	Tower Hamlets	E09000030	295	26	266	0.902	284300.0	21.6	12564.7	45720

```
1 print("The shape of the df (rows x columns) is:", borough_context.shape)  
2 print("The names of each column are:", list(borough_context.columns)) #I used list here just to make the output slightly neater to
```

The shape of the df (rows x columns) is: (33, 20)

The names of each column are: ['borough_name', 'borough_code', 'venues', 'venue_types', 'venues_with_websites', 'website_share', 'population_2015', 'area_sq_km', 'population_density_2013', 'median_household_income_2012_13', 'median_house_price_2014', 'ptal_score_2014', 'pct_bame_2011', 'pct_not_born_uk_2011', 'pct_level4_qualifications_2011', 'crime_rate_2014_15', 'pct_area_open_space_2014', 'venues_per_10k_residents', 'venues_per_sq_km', 'venue_types_per_100k_residents']

`pd.read_csv(filepath)` reads a CSV file into a DataFrame. The main argument is the file path. **More on this later!**

`head()` shows the first rows; `shape` gives (rows, columns); `columns` gives the column names.

What is in it?

```
1 borough_context.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 33 entries, 0 to 32
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   borough_name                          33 non-null    str
1   borough_code                          33 non-null    str
2   venues                               33 non-null    int64
3   venue_types                           33 non-null    int64
4   venues_with_websites                  33 non-null    int64
5   website_share                         33 non-null    float64
6   population_2015                       33 non-null    float64
7   area_sq_km                            33 non-null    float64
8   population_density_2013                33 non-null    float64
9   median_household_income_2012_13       33 non-null    float64
10  median_house_price_2014                33 non-null    float64
11  ptal_score_2014                        33 non-null    float64
12  pct_bame_2011                          33 non-null    float64
13  pct_not_born_uk_2011                   33 non-null    float64
14  pct_level4_qualifications_2011         33 non-null    float64
15  crime_rate_2014_15                     33 non-null    float64
16  pct_area_open_space_2014               33 non-null    float64
```

`info()` prints the structure of the DataFrame: row count, column names, non-null counts and data types. It does not take a required parameter.

Statistical summary

```
1 borough_context[[
2     "venues",
3     "population_2015",
4     "venues_per_10k_residents",
5     "venues_per_sq_km",
6     "median_household_income_2012_13",
7     "ptal_score_2014",
8  ]].describe().round(1)
```

	venues	population_2015	venues_per_10k_residents	venues_per_sq_km	median_household_income_2012_13	ptal_score_2014
count	33.0	33.0	33.0	33.0	33.0	33.0
mean	134.4	261601.5	9.3	6.2	52520.3	4.0
std	145.1	74550.7	23.7	9.0	15916.5	1.3
min	22.0	8100.0	0.9	0.2	34080.0	2.4
25%	48.0	220800.0	1.5	0.6	44430.0	3.0
50%	84.0	269100.0	2.8	1.8	45860.0	3.4
75%	141.0	320100.0	7.9	9.2	56920.0	4.9
max	653.0	393200.0	137.0	34.7	116350.0	7.9

`describe()` summarises numeric columns. `round(1)` rounds the displayed numbers to one decimal place; the parameter is the number of decimal places.

Part III: Files and loading CSVs

You asking me “Where is the file I downloaded on my computer?” is similar to asking me “Where do I keep my forks and knives?” **I don’t know!** You should know. Learn the structure of your PC.

Where your files are

Absolute path (full address):

- Windows: `C:\Users\Mark\Documents\projects\7aavbcs5\data.csv`
- Mac/Linux: `/Users/Mark/Documents/projects/7aavbcs5/data.csv`

Relative path (from where you are):

- `data.csv` – in the current folder
- `../data.csv` – in the folder above
- `data/raw.csv` – in a subfolder

```
7AAVBCS5/  
├── weeks/  
│   ├── week-03/  
│   │   └── w03_worksheet.ipynb  
└── data/  
    └── london_cultural_venues_2023.csv
```

What is a CSV?

- Plain text file storing tabular data
- Each line is a row
- Columns separated by commas
- Human-readable and universal

Also TSV files: tab-separated, same idea, better when the data itself contains commas.

```
name,age,city
Alice,25,London
Bob,30,Manchester
Mo,45,Cardiff
Carol,28,Edinburgh
Zhao,75,Oxford
```

Five records, each one person: name, age, city.

Reading a CSV

```
1 df = pd.read_csv("london_cultural_venues_2023.csv")
2
3 df = pd.read_csv(
4     "london_cultural_venues_2023.csv",
5     sep=";",          # what separates the columns
6     encoding="utf-8",  # how the text is stored
7     na_values=["", "NA"], # what counts as missing
8 )
```

Always check after loading: `df.info()`, `df.head()`, `df.describe()`

Useful `read_csv()` parameters: `sep` says how columns are separated, `encoding` says how text is stored, and `na_values` lists strings that should be treated as missing.

If loading fails with a `UnicodeDecodeError`, the file is not UTF-8. Try `encoding="cp1252"`.

Part IV: Selecting rows and columns

Getting a column

```
1 borough_context["borough_name"].head()
```

```
0    Westminster
1      Camden
2     Hackney
3    Southwark
4  Tower Hamlets
Name: borough_name, dtype: str
```

Two ways, and one is safer:

```
1 borough_context["borough_name"]    # always works
2 borough_context.borough_name        # only if the name has no spaces and is not a method name
```

`data["column_name"]` selects one column by name. This is bracket selection, not a function: the value inside the brackets is the column label.

Selecting several columns

```
1 borough_context[[
2     "borough_name",
3     "venues",
4     "venues_per_10k_residents",
5     "median_household_income_2012_13",
6 ]].head(3)
```

	borough_name	venues	venues_per_10k_residents	median_household_income_2012_13
0	Westminster	653	28.32	80760.0
1	Camden	469	20.36	67990.0
2	Hackney	403	15.34	42690.0

The inner brackets are a **list** of column names – which is why there are two sets.

`data[[column_list]]` selects several columns. The key input is a list of column names, such as `["borough_name", "venues"]`. `head(3)` uses the optional `n` parameter to show three rows instead of the default five.

Rows and columns together

So far we have selected whole columns.

Sometimes we want one value, or a small part of the table:

- a **row**
- a **column**
- the value where a row and column meet (the **cell**)

pandas gives us two tools for this:

Tool	Use it when...	Example
<code>.loc[]</code>	you know the row or column label	<code>borough_context.loc[0, "borough_name"]</code>
<code>.iloc[]</code>	you know the row or column position number	<code>borough_context.iloc[0, 0]</code>
<pre>1 print(borough_context.loc[0, "borough_name"]) 2 print(borough_context.iloc[0, 0])</pre>		
<pre>Westminster Westminster</pre>		

Rows and columns both go inside the square brackets: first the row, then the column.

Slices of rows

```
1 borough_context.iloc[0:3]
```

	borough_name	borough_code	venues	venue_types	venues_with_websites	website_share	population_2015	area_sq_km	population_density_2013	media
0	Westminster	E09000033	653	24	578	0.885	230550.0	22.0	10285.8	80760
1	Camden	E09000007	469	26	369	0.787	230300.0	21.8	10482.1	67990
2	Hackney	E09000012	403	26	346	0.859	262700.0	19.1	13471.7	42690

With `.iloc[start:stop]`, the slice uses integer positions and the `stop` position is excluded. **Note:** By default, if you do not provide a second number or slide with a comma (`,`), `.iloc()` will assume you want rows.

`.iloc[0:3]` gives three rows – the end is excluded, like a list slice. `.loc[0:3]` gives **four** – label slices *include* the end. I'm sorry.

What is a mask?

A **mask** is a True/False column we use to decide which rows to keep.

```
1 mask = borough_context["venues_per_10k_residents"] > 10
2 mask.head()
```

```
0    True
1    True
2    True
3   False
4    True
Name: venues_per_10k_residents, dtype: bool
```

Each row gets one answer:

- **True** means keep this row
- **False** means leave this row out

```
1 borough_context[mask][["borough_name", "venues_per_10k_residents"]].head()
```

	borough_name	venues_per_10k_residents
0	Westminster	28.32
1	Camden	20.36
2	Hackney	15.34
4	Tower Hamlets	10.38
6	Islington	10.33

The condition makes the mask. Putting the mask inside `data[...]` filters the rows.

Selecting by condition

But what if we don't know what data it is we want? For example, we only want the rows where the population is over 100,000 or the number of venues is below 50?

```
1 # What boroughs have high number of venues per resident to access?
2 high_access = borough_context[
3     borough_context["venues_per_10k_residents"] > 10
4 ]
5 high_access[["borough_name", "venues_per_10k_residents"]]
```

	borough_name	venues_per_10k_residents
0	Westminster	28.32
1	Camden	20.36
2	Hackney	15.34
4	Tower Hamlets	10.38
6	Islington	10.33
10	City of London	137.04

The bit inside the brackets is the True/False array from last week – one per row. pandas keeps the rows where it is `True`.
(I'll show this when live coding)

`data[condition]` filters rows. The condition must produce one `True` or `False` value for every row.

Combining conditions

And we can combine our filters for more complex masks.

```
1 # What boroughs have high number of venues per resident to access AND a income of less than £50k (in 12/13 GBP)?
2 subset = borough_context[
3     (borough_context["venues_per_10k_residents"] > 10) &
4     (borough_context["median_household_income_2012_13"] < 50000)
5 ]
6 subset
```

	borough_name	borough_code	venues	venue_types	venues_with_websites	website_share	population_2015	area_sq_km	population_density_2013	media
2	Hackney	E09000012	403	26	346	0.859	262700.0	19.1	13471.7	42690
4	Tower Hamlets	E09000030	295	26	266	0.902	284300.0	21.6	12564.7	45720

& combines two column-wise conditions; **each** condition must be wrapped in brackets.

Use & and |, not and and or – and **every condition needs its own brackets.**

The readable alternative

A more readable way of subsetting is the query method.

```
1 borough_context.query(  
2     "venues_per_10k_residents > 10 and median_household_income_2012_13 < 50000"  
3 ).shape
```

```
(2, 20)
```

`.query()` takes a condition as text. No masks, no ampersands, no bracket-counting.

`query(expr)` filters rows using a string expression. The key parameter is `expr`: the condition written as text using column names.

Finding the biggest and smallest

```
1 biggest_value = borough_context["venues_per_10k_residents"].max()
2 print(biggest_value)
```

```
137.04
```

But which one is it? This is where we can use `df["column"].idxmax()`

```
1 row_label = borough_context["venues_per_10k_residents"].idxmax()
2 borough_context.loc[row_label, [
3     "borough_name",
4     "venues",
5     "population_2015",
6     "venues_per_10k_residents",
7 ]]
```

```
borough_name      City of London
venues              111
population_2015    8100.0
venues_per_10k_residents  137.04
Name: 10, dtype: object
```

`max()` returns the largest **value**. `idxmax()` returns the **row label** where that largest value appears. Use that label with `.loc` to retrieve the whole row. The same operations work with `min()` and `idxmin()`

Adding a column

```
1 borough_context["above_average_venue_access"] = (  
2     borough_context["venues_per_10k_residents"] >  
3     borough_context["venues_per_10k_residents"].mean()  
4 )  
5  
6 borough_context[[  
7     "borough_name",  
8     "venues_per_10k_residents",  
9     "above_average_venue_access",  
10  ]].head()
```

	borough_name	venues_per_10k_residents	above_average_venue_access
0	Westminster	28.32	True
1	Camden	20.36	True
2	Hackney	15.34	True
3	Southwark	9.95	True
4	Tower Hamlets	10.38	True

Assigning to a name that does not exist creates it. Assigning to one that does **overwrites** it.

`mean()` calculates the average of a numeric Series. Here it gives the threshold for a new True/False column.

Best practice: keep the original data intact when you can. Create derived columns deliberately and use clear names.

Writing a CSV

```
1 camden_venues.to_csv("camden_venues.csv", index=False)
```

`to_csv(path, index=False)` writes a DataFrame to a CSV file. `path` is the output filename; `index=False` stops pandas writing row labels as an extra column.

`index=False` matters. Without it, pandas writes the row numbers as an extra unnamed column – and if you then read that file back, you get a mysterious `Unnamed: 0`.

File practice: best and good

Important!

1. **Keep raw data separate** – never overwrite original data files
2. **Do not depend on the working directory** – use paths that work anywhere
3. **Version control** – date your output:
`analysis_2026_10_22.csv`
4. **Consistent naming** – good:
`lowercase_with_underscores.csv`; bad: `My Data File (Final) v2.CSV`

Good practice

1. **Handle missing values** – know how they appear in your data
2. **Document your data** – keep a README explaining the columns
3. **Check before you write** – do not silently overwrite

Common data frame errors

Error	Means
<code>KeyError: 'Borough_name'</code>	No such column. Names are case sensitive – it is <code>borough_name</code>
<code>ValueError: Length of values does not match length of index</code>	The column you are adding is the wrong length
<code>UnicodeDecodeError</code>	The file is not UTF-8
<code>ValueError: The truth value of a Series is ambiguous</code>	You used <code>and</code> where you needed <code>&</code>

Debugging strategies

1. **Check what it is** – `type(my_data)`
2. **Verify column names** – `my_data.columns`. Exact spelling matters
3. **Check dimensions** – `my_data.shape`. Do rows and columns match what you expect?
4. **Look for missing values** – `my_data.isna().sum()`
5. **Verify the file exists** – `Path("data.csv").exists()`
6. **Look at it** – click the data frame in Positron's Variables pane

Debugging with AI

When you paste an error into <https://ai.create.kcl.ac.uk/chat>, include **three things**: what you were trying to do, the code, and the **full** error message.

Ask it to *explain* the error, not just to fix your code. You will need to recognise the same error next week, in a closed quiz, without it.

Use the KCL service, not a personal account: anything pasted into a consumer AI tool may be retained and used for training. That matters when it is your research data.

Part V: Live Coding

Creating a data frame

Requirements:

- `name` (text)
- `borough` (text)
- `venue_type` (text)
- `visited` (True/False)

Steps:

1. Build the dictionary of columns
2. Make the data frame
3. Explore with `shape`, `columns`, `info()` and `head()`

Exploring the London borough data

1. **Basic exploration** – one row per borough, meaningful numeric variables
2. **Columns and filters** – venue access, income and transport accessibility
3. **Rows and labels** – why filtering changes `.loc/.iloc`
4. **Meaningful extremes** – venues per resident and venues per square kilometre

File practice (raw and aggregated)

1. **Read the borough context file** – one row per borough
2. **Read the venue file** – one row per venue
3. **Compare** – borough-level context versus venue-level records
4. **Modify and save** – high-access boroughs, saved with `index=False`

Thanks!

 Note

The quiz in next week's lecture covers **this week's** material.